GPUs are scarce & expensive.

Strategies for GPU workers in Buildbarn (and Bazel)

Chris McDonald

Notes & caveats

01 Infrastructure tech lead for DoorDash Labs

02 Linux + AWS + NVIDIA

03 I am not a Kubernetes expert.

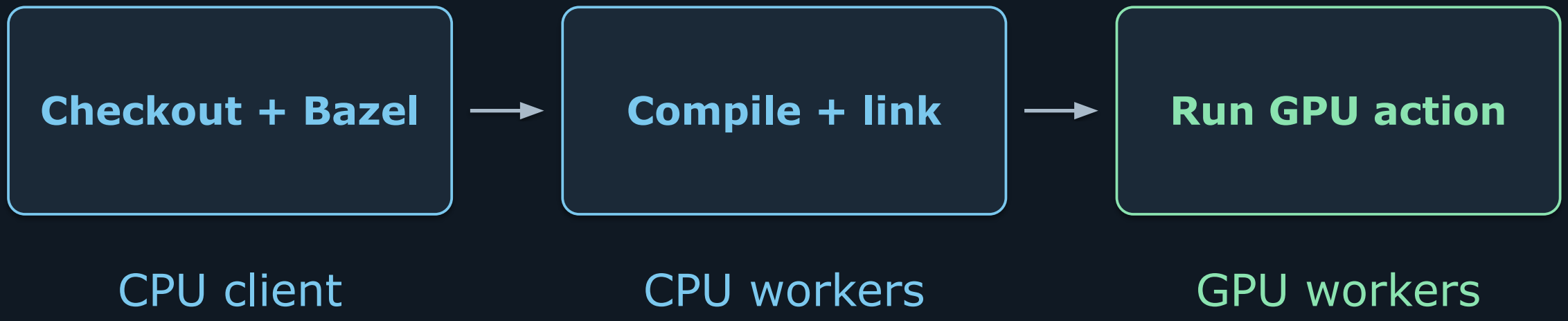
Talk overview

01 Bazel stuff: platforms, rules, and CUDA.

02 Buildbarn stuff: Queues and workers.

03 k8s stuff: Pods, resources, etc.

The GPU should be busy doing GPU work



Idle GPUs are the enemy!

Describe a capability: has_gpu

```
constraint_value(
    name = "has_gpu",
    constraint_setting = ":gpu",
)
```

CPU first. GPU when requested.

```
# remote_cpu
constraint_values = [":no_gpu"]

# remote_gpu
constraint_values = [":has_gpu"]
exec_properties = {"gpu": "True"}
```

Register in order:
remote_cpu
remote_gpu

Add the common OS / CPU constraints to both platforms.

Split targets at the GPU boundary



Separate CPU preparation from GPU execution.

Rules can contain both CPU and GPU actions

```
"gpu": exec_group(
    exec_compatible_with = [
        "//platforms:has_gpu",
    ],
)
```

Inside _impl:

```
ctx.actions.run(
    exec_group="gpu",
)
```

Use exec_groups to split up actions within the implementation function of a single rule.

Make the user API boring

```
requires_gpu = True
```

Let the wrapper handle the Bazel esoterica.

Bazel sharp edges: property scope

```
# Too broad
exec_properties = {
    "gpu": "True",
}
```

```
# Test action only
exec_properties = {
    "test.gpu": "True",
}
```

One missing prefix can send compilation to GPU workers.

Bazel sharp edges: hidden GPU work

Coverage postprocessing

Split spawns can inherit the test's GPU placement.

Collect on GPU; postprocess on CPU.

Bazel 9 test toolchains

Default matches every target-platform constraint.

Match OS/CPU; let tests request GPUs individually.

Bazel 9: incompatible_use_default_test_toolchain is enabled by default.

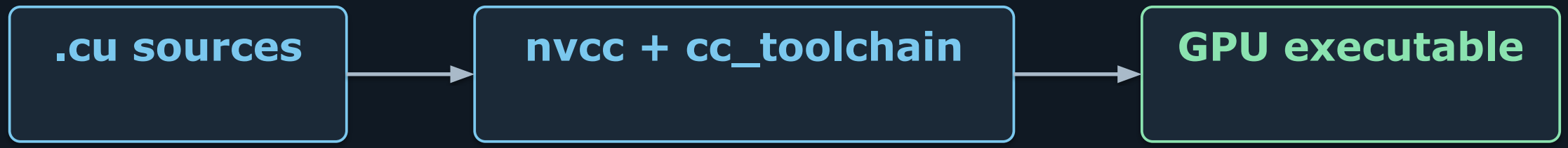
Bazel ≤ 8: hide that prefix in the macro

```
if requires_gpu:
    props["test.gpu"] = "True"
```

Bazel 9: constrain the test execution group

```
if requires_gpu:
    groups["test"] = groups.get("test", []) + [
        "//platforms:has_gpu",
    ]
```

rules_cuda gets the big stuff right



Build + link on CPUs

Use the link-time libcuda.so stub

Real driver at runtime

Cross-compile by composing toolchains

Component	Chosen for
nvcc + compiler helpers	Execution platform
CUDA CRT + target libraries	Target platform
cc_toolchain output	Target CPU / ABI

“Host code” inside CUDA still needs the target architecture.

Build for the union of targets

AWS family	GPU / compute capability	Baseline code
G5	A10G / 8.6	sm_86
G6 / G6e	L4 / L40S / 8.9	sm_86 (compatible)
G7e	RTX PRO 6000 / 12.0	sm_120

cloud-x86_64 → sm_86 + sm_120; validate performance separately.

Bazel 8+: keep CUDA arch flags on the platform

```
flags = [
    "--@rules_cuda//cuda:archs=sm_86;sm_120",
]
```

One abstract target platform; one intentionally chosen compatibility set.

Good rules. Room to improve.

01 Device LTO + header-only CUDA targets: a limitation I hit

02 One coarse nvcc action hides multiple compilation phases



More sm targets → larger actions, instead of more schedulable actions.

Now, the Buildbarn side.

How do we keep scarce GPU workers busy?

Queues → whole GPUs → MPS → DRA

Buildbarn queues: match the contract

```
{ name: "gpu", value: "True" }
```

Bazel selects an exec platform before available workers are known.

Bin compatible GPU models together; keep worker sizes out of the platform.

The “naive” approach

```
resources:
  limits:
    nvidia.com/gpu: 1
```

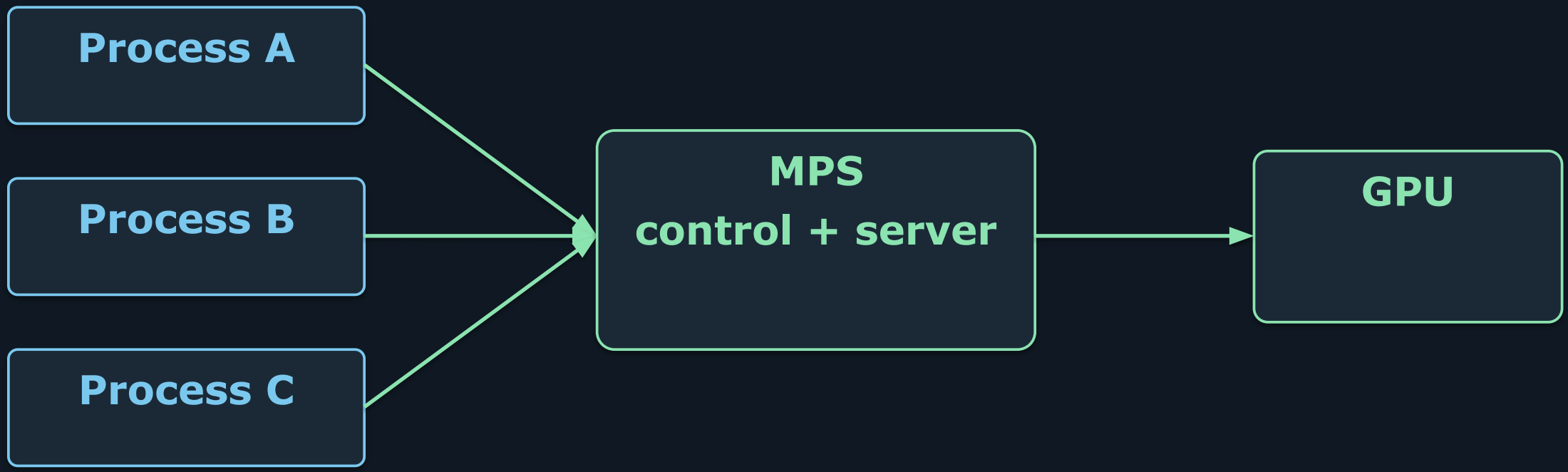
A tiny CUDA test
gets the same unit
as a huge GPU workload.

1,000 GPU actions?

Simply spin up 1,000 GPU instances.

What could possibly go wrong?

NVIDIA MPS: Multi-Process Service



Concurrent work can fill gaps left by a single process.

MPS is cooperative sharing

01 SM limits are not a promise of “exactly 20% throughput.”

02 A memory cap is PER PROCESS.

03 Several processes ≠ one aggregate action budget.

Useful utilization control; a weak isolation boundary.

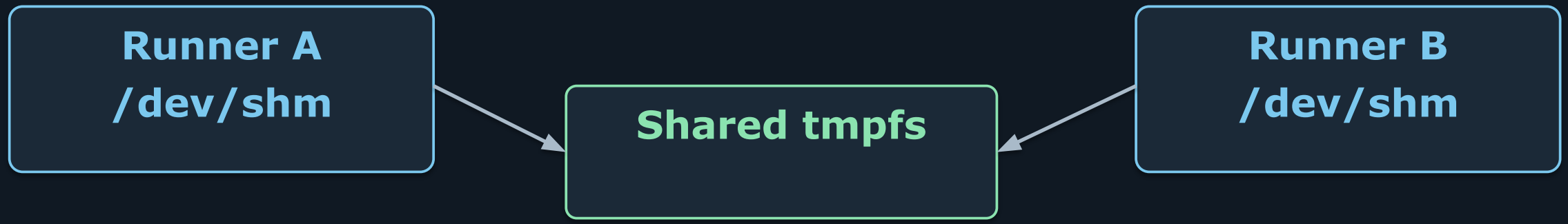
The device plugin makes MPS easy to try

```
mps:
  renameByDefault: true
  resources:
  - name: nvidia.com/gpu
    replicas: 5
```

```
nvidia.com/gpu.shared: 1
```

half = 2 replicas / small = 5 / tiny = 10

MPS sharp edges: shared /dev/shm



Name collisions. Cleanup collisions.

A test environment that is no longer private.

You can fork the plugin. But… ew.

A “shared GPU” is a property of the node

HALF POOL	SMALL POOL	TINY POOL
2 replicas / GPU	5 replicas / GPU	10 replicas / GPU

Same resource name represents different portions of a GPU.

Rebalancing pools can mean losing the machine



DRA: request devices, then share the claim



Count + identity + an explicit reference

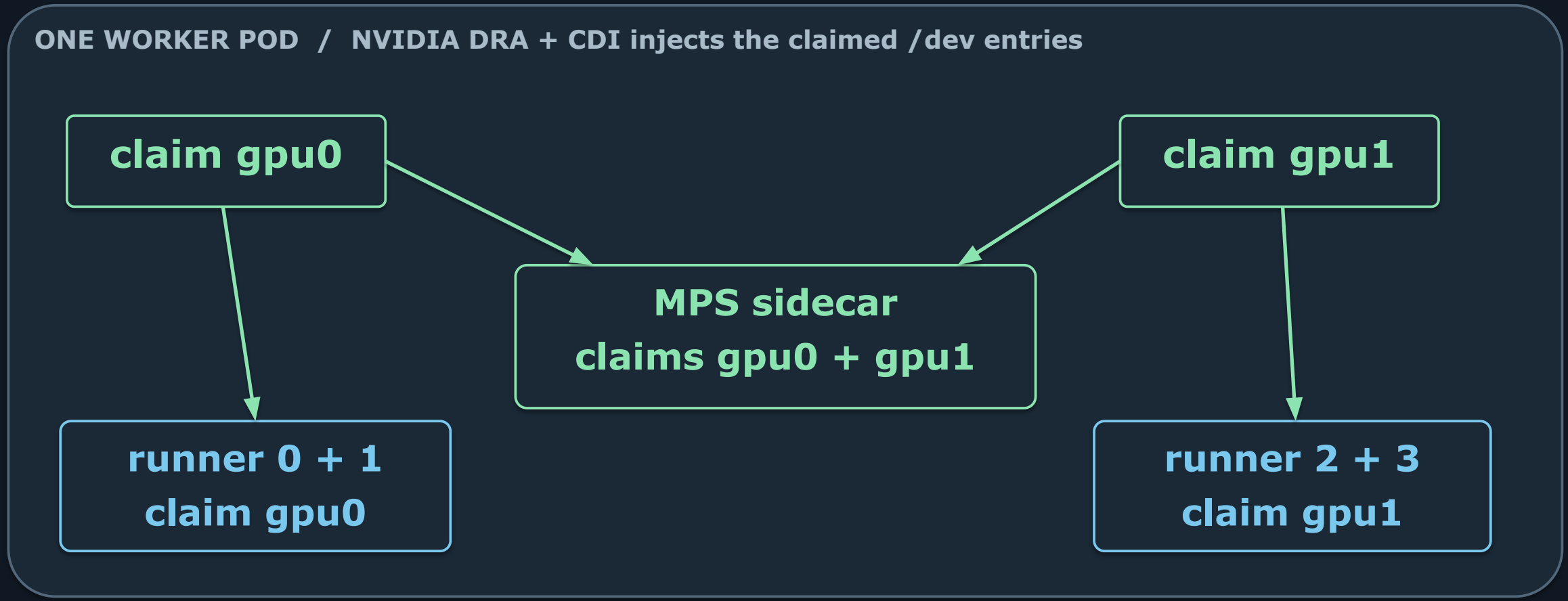
NVIDIA DRA Driver for GPUs replaces device-plugin allocation.

One claim template; one device per claim

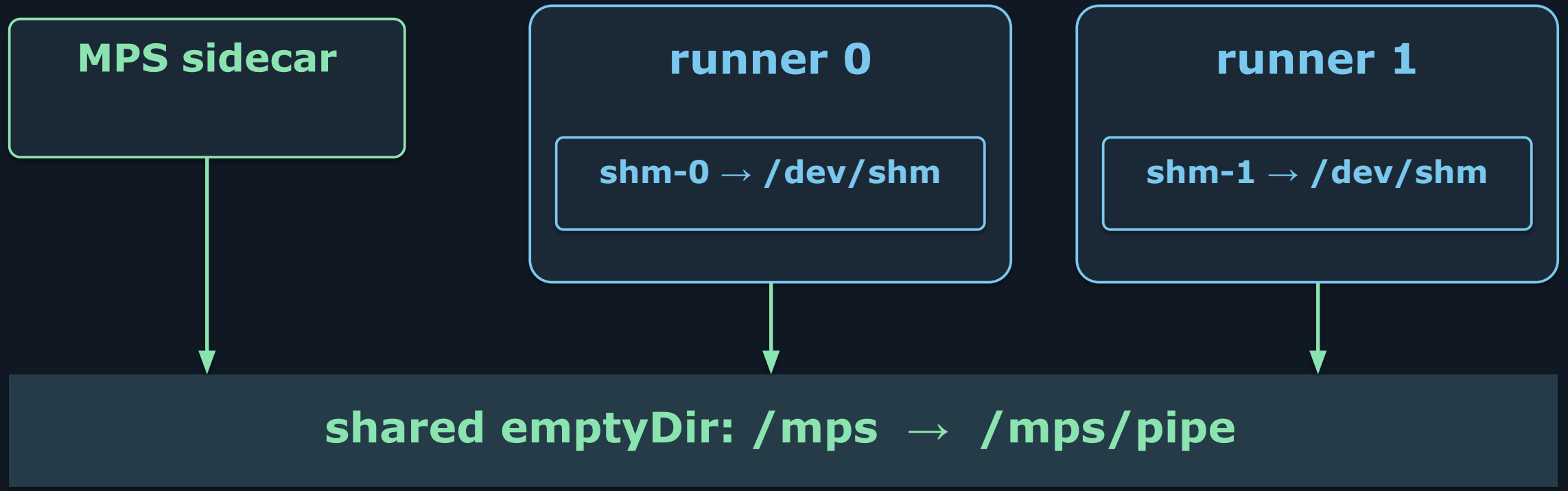
```
exactly:
  deviceClassName: gpu.nvidia.com
  count: 1
```

Claim template:
worker-gpu

Device wiring: one claim, several containers



Volume wiring: share the pipe, keep SHM private



Private SHM stays inside each runner.

Start MPS before accepting actions

```
initContainers:
- name: mps
  restartPolicy: Always
```

```
# startupProbe check
test -p /mps/pipe/control
```

startupProbe gates runner startup; readinessProbe alone does not.

Own the MPS configuration inside the Pod

```
nvidia-cuda-mps-control -d

echo "set_default_active_thread_percentage 20" |
  nvidia-cuda-mps-control
```

Configure once per worker Pod; all its runners use the same pipe.

Carry the pipe path into the action environment

```
# Action environment
CUDA_MPS_PIPE_DIRECTOR
Y:
  "/mps/pipe"
```

Shared /mps
Private /dev/shm
Claim gpu0

Container environment ≠ action environment. Make the handoff explicit.

Generate worker shapes, not node-pool types

```
runners_per_gpu = {
  full  = 1
  half  = 2
  small = 5
  tiny  = 10
}
```

Worker size is a Pod choice now

ONE COMPATIBLE GPU NODE POOL

full	half	small	tiny

Buildbarn sizeClass: full 10 / half 5 / small 2 / tiny 1

Rebalance worker Pods while retaining scarce GPU instances.

Bonus: a bridge from numerical GPU resources

```
resources:
  limits:
    nvidia.com/gpu: 1
```

Useful migration bridge; explicit claims still describe shared identity.

Questions?

01 Keep CPU work off scarce GPU workers.

02 Keep compatible devices in broad queues.

03 Let Pods own sharing policy; let DRA own device allocation.

Backup / public references: Bazel + CUDA

Bazel execution groups

Platform flags

rules_cuda documentation

NVIDIA compiler / binary compatibility

CUDA compute capabilities

AWS accelerated instance specifications

Backup / public references: MPS + DRA

NVIDIA device plugin

NVIDIA MPS architecture

MPS v2 commands + per-process limits

Kubernetes Dynamic Resource Allocation

NVIDIA DRA Driver for GPUs

Kubernetes native sidecars

PRESENTER NOTES

Backup / details to pin before the talk

01 Pin versions for the /dev/shm and device-LTO anecdotes.

02 Newer NVIDIA DRA drivers can also manage MPS.

03 The snippets show the design; production manifests need the rest.